

MATH405-HW5-Q1

November 27, 2025

Question 1(a):

(a)

$$u_{xx} - u = f(x)$$

$\Omega = [0, 1], u'(0) = 0,$

$(u_0) \quad (u_2)$

$x_0 \quad x_1 \quad x_2$

$x_1 = 0, u'(x_1) = 0$

$$u'(x_1) = \frac{u_2 - u_0}{2h} = 0 \Rightarrow u_0 = u_2$$

Hence $\frac{u_2 - 2u_1 + u_0}{h^2} - u_1 = f(0)$

which means the first row of the linear system becomes

$$\left(\frac{2}{h^2} - 1\right)u_1 + \left(\frac{2}{h^2}\right)u_2 = f(0)$$

Question 1(b):

I try to use $u(x) = \cos(2\pi x)$, then $u'(0) = 0$ is true and $u(1) = 1$ is true. In this setup, $f(x) = u_{xx} - u = -(4\pi^2 + 1)\cos(2\pi x)$. Boundary condition at $x = 1$: As clarified on Piazza, I did not use a ghost point at the right boundary and I will explain this: Instead, I directly imposed the Dirichlet condition by replacing the last equation with $u_{N+1} = 1$.

```
[5]: import numpy as np
import matplotlib.pyplot as plt
# Directly copy the code from our Q6 in HW3, but modify the A matrix.

# Exact solution u(x) and f(x)
def u_exact(x):
    return np.cos(2* np.pi * x)

def f(x):
    return -(4 * np.pi**2 + 1)*np.cos(2* np.pi * x)
```

```

def build_A_ghost(N):
    h = 1.0 / N
    n = N + 1 # size
    A = np.zeros((n, n))
    # left boundary row
    A[0,0] = -2.0/h**2 - 1.0
    A[0,1] = 2.0/h**2
    # interior rows j=1,...,N-1
    for j in range(1, N):
        A[j, j-1] = 1.0/h**2
        A[j, j] = -2.0/h**2 - 1.0
        A[j, j+1] = 1.0/h**2
    # right boundary row j=N
    A[N, N-1] = 0.0/h**2
    A[N, N] = 1.0
    return A, h

# Setup
N_values = [4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096]
h_values = []
err_values = []

# Results
for k, N in enumerate(N_values):
    A, h = build_A_ghost(N)
    x = np.linspace(0, 1, N+1)
    F = f(x)
    # BC
    F[N] = 1
    U = np.linalg.solve(A, F) # AU=F
    err = np.abs(U - u_exact(x)) # |U-u|
    err_max = np.max(err)
    h_values.append(h)
    err_values.append(err_max)

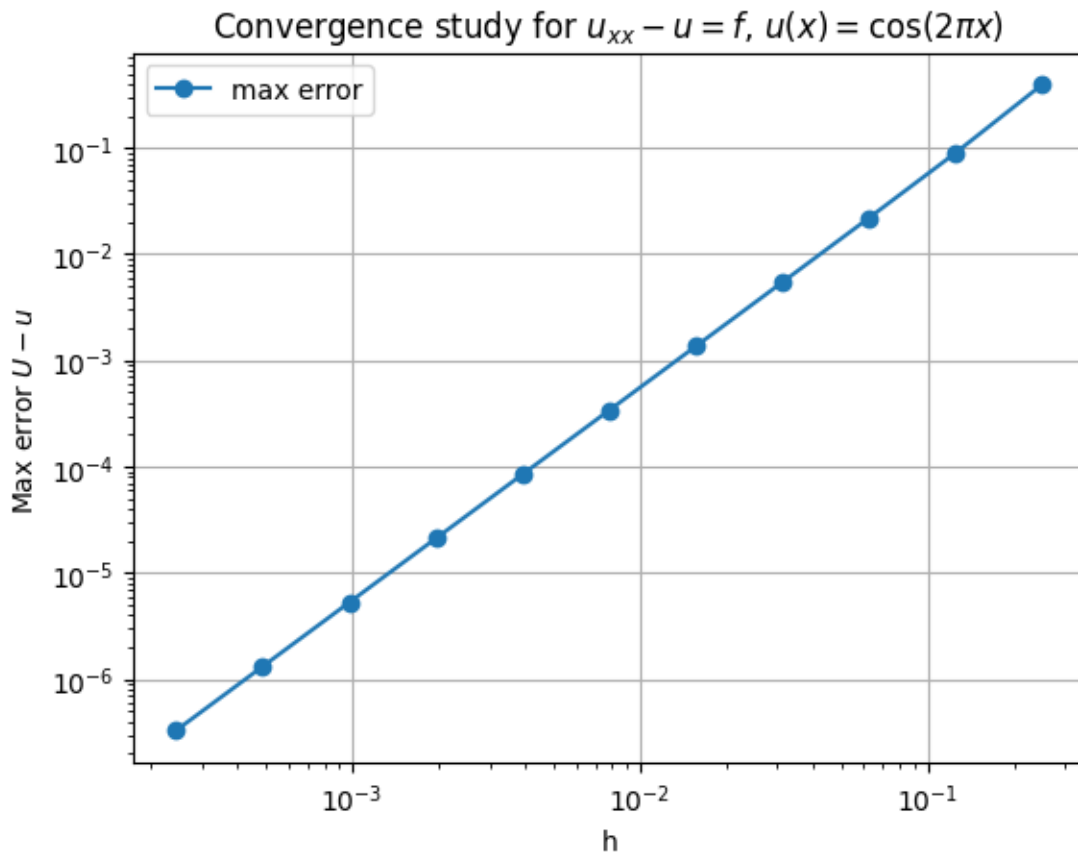
    if k == 0:
        print(f"n={N}  h={h:.12f}  max|error|={err_max:.12f}  order=---")
    else:
        rate = np.log(err_values[-2] / err_values[-1]) / np.log(2.0) # p =  $\frac{\ln(E1 / E2)}{\ln(2)}$ 
        print(f"n={N}  h={h:.12f}  max|error|={err_max:.12f}  order={rate:.12f}")

# log-log plot
plt.figure()
plt.loglog(h_values, err_values, marker="o", label="max error")
plt.xlabel("h")

```

```
plt.ylabel(r"Max error $U - u$")
plt.title(r"Convergence study for $u_{xx} - u = f$, $u(x)=\cos(2\pi x)$")
plt.grid(True)
plt.legend()
plt.show()
```

n=4	h=0.250000000000	max error =0.392450485655	order=--
n=8	h=0.125000000000	max error =0.089409610351	order=2.134008829322
n=16	h=0.062500000000	max error =0.021854731670	order=2.032484245386
n=32	h=0.031250000000	max error =0.005433246024	order=2.008059384497
n=64	h=0.015625000000	max error =0.001356850946	order=2.001552130667
n=128	h=0.007812500000	max error =0.000339271941	order=1.999748219776
n=256	h=0.003906250000	max error =0.000084810601	order=2.000125605784
n=512	h=0.001953125000	max error =0.000021202189	order=2.000031403070
n=1024	h=0.000976562500	max error =0.000005300566	order=1.999994845578
n=2048	h=0.000488281250	max error =0.000001325140	order=2.000001638771
n=4096	h=0.000244140625	max error =0.000000331283	order=2.000010316277



Hence, this method is convergence ($h \rightarrow 0$, $error \rightarrow 0$). As shown in the results above, the order of accuracy is 2.

MATH405-HW5-Q2

November 25, 2025

Math 405 HW5 Q2

Consider $u''(x) = -\lambda u(x)$, $x \in [0, 2\pi]$, λ is constant.

This is equivalent to $u''(x) + \lambda u(x) = 0$.

General solutions:

I $\lambda < 0$: let $\lambda = -\mu^2$ with $\mu > 0$, then $u'' - \mu^2 u = 0 \Rightarrow u(x) = Ae^{\mu x} + Be^{-\mu x}$

The exp function cannot satisfy the three types of BCs.

Hence $\lambda < 0$ gives only the trivial eigenfunction $u \equiv 0$.

II $\lambda = 0$: $u'' = 0 \Rightarrow u(x) = Ax + B$

(a) periodic: $u(0) = u(2\pi) \Rightarrow A = A + 2\pi B \Rightarrow B = 0$

$u'(0) = u'(2\pi) \Rightarrow B = B$

Hence $u(x) \equiv A$ when $A \neq 0$ gives non-trivial solution

$\Rightarrow \lambda_0 = 0, u_0(x) = \text{constant}$.

(b) zero-Dirichlet: $u(0) = 0 \Rightarrow A = 0$, $u(2\pi) = 0 \Rightarrow A + 2\pi B = 0 \Rightarrow B = 0$

only trivial solution.

(c) zero-Neumann: $u'(x) = B$, $u'(0) = 0 \Rightarrow B = 0$ then $u'(2\pi) = 0$.

$u(x) \equiv A$ when $A \neq 0$ gives non-trivial solution.

$\Rightarrow \lambda_0 = 0, u_0(x) = \text{constant}$.

III $\lambda > 0$: let $\lambda = \mu^2$ ($\mu > 0$), then $u'' + \mu^2 u = 0 \Rightarrow u(x) = A \cos(\mu x) + B \sin(\mu x)$

(a) periodic: $u(0) = u(2\pi) \Rightarrow u(0) = A = u(2\pi) = A \cos(2\pi\mu) + B \sin(2\pi\mu)$

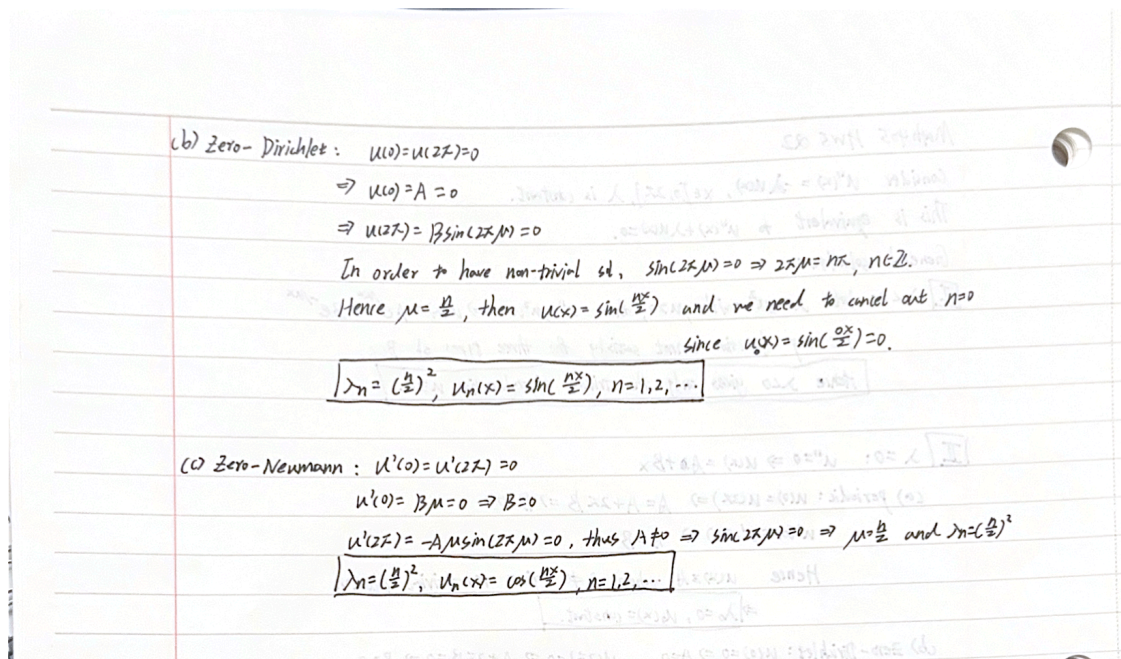
$u'(0) = u'(2\pi) \Rightarrow u'(0) = B\mu = u'(2\pi) = -A\mu \cos(2\pi\mu) + B\mu \sin(2\pi\mu)$

Hence $\begin{pmatrix} A \\ B \end{pmatrix} \neq \begin{pmatrix} 1 - \cos(2\pi\mu) & -\sin(2\pi\mu) \\ \sin(2\pi\mu) & 1 - \cos(2\pi\mu) \end{pmatrix} \begin{pmatrix} A \\ B \end{pmatrix} = 0$

We need non-trivial sol so $\det(\uparrow) = (1 - \cos(2\pi\mu))^2 + (\sin(2\pi\mu))^2 = 0$

$\Rightarrow$ only when $1 - \cos(2\pi\mu) = 0$ and $\sin(2\pi\mu) = 0$ Hence $2\pi\mu = 2\pi n$, $n \in \mathbb{Z}$.

$\lambda_n = n^2$, $u_n(x) = \cos(nx), \sin(nx)$, $n = 1, 2, \dots$



Summary:

- (a) Periodic BC

$$\lambda_n = n^2, \quad u_n(x) = \cos(nx), \sin(nx), \quad n = 0, 1, 2, \dots$$

Here $n = 0$ corresponds to the constant solution, with $\lambda_0 = 0$.

- (b) Zero-Dirichlet BC

$$\lambda_n = \left(\frac{n}{2}\right)^2, \quad u_n(x) = \sin\left(\frac{n\pi x}{2}\right), \quad n = 1, 2, 3, \dots$$

- (c) Zero-Neumann BC

$$\lambda_n = \left(\frac{n}{2}\right)^2, \quad u_n(x) = \cos\left(\frac{n\pi x}{2}\right), \quad n = 0, 1, 2, 3, \dots$$

Here $n = 0$ corresponds to the constant solution, with $\lambda_0 = 0$.

MATH405-HW5-Q3

November 26, 2025

Question 3(a):

$$u''(x) + \lambda u(x) = 0$$

I used `scipy.sparse.linalg.eigs` in Python instead of Matlab's `eigs`.

```
[10]: import numpy as np
import matplotlib.pyplot as plt
from scipy.sparse import diags
from scipy.sparse.linalg import eigs

def lambda_exact(n):
    return (0.5 * n)**2

def u_exact(x, n):
    return np.sin(0.5 * n * x)

# Build A matrix for u'' + lambda u = 0 with zero-Dirichlet BC
# last homework option 2 matrix
def build_A(N):
    L = 2.0 * np.pi
    h = L/N
    A = np.zeros((N+1,N+1))
    for j in range(1, N):
        A[j, j-1] = 1.0/h**2
        A[j, j] = -2.0/h**2
        A[j, j+1] = 1.0/h**2
    A[0, 0] = -2.0/h**2
    A[N, N] = -2.0/h**2
    return A, h

N = 64
A, h = build_A(N)
L = 2.0 * np.pi

vals, vecs = eigs(A, k=24, which="SM")
idx = np.argsort(np.abs(vals))
vals = vals[idx]
vecs = vecs[:, idx]
```

```

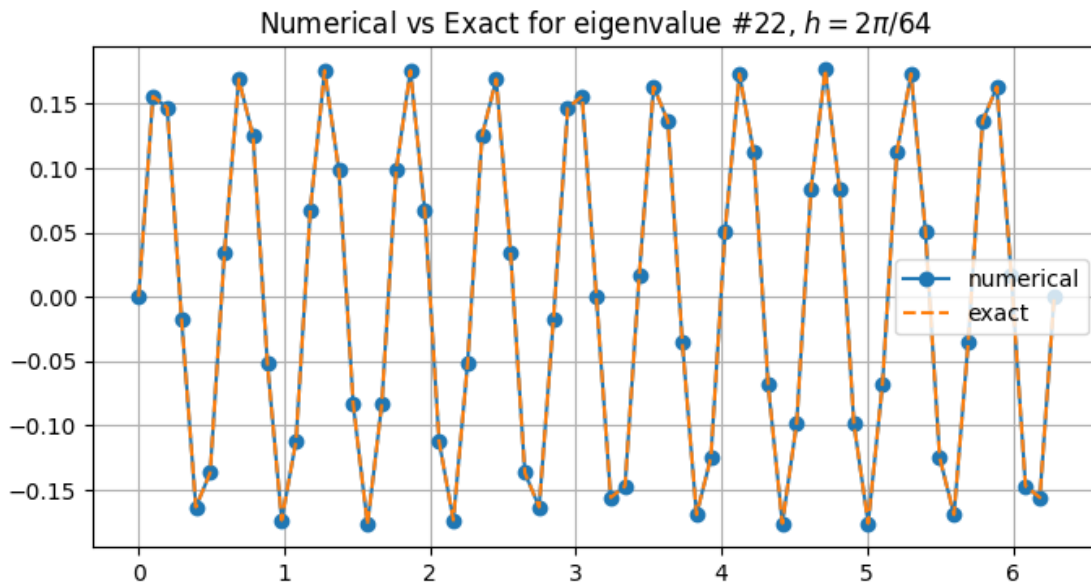
k = 22 #22nd
lam_num = np.real(vals[k-1])
u_num = np.real(vecs[:, k-1])

x = np.linspace(0.0, L, N+1)
u_ex = u_exact(x, k)

# scale the constant, the "if" is make sure that we don't inverse the constant
if u_num[1] > 0.0:
    u_ex *= np.max(np.abs(u_num))/np.max(np.abs(u_ex))
else:
    u_ex *= -np.max(np.abs(u_num))/np.max(np.abs(u_ex))

plt.figure(figsize=(8,4))
plt.plot(x, u_num, "o-", label="numerical")
plt.plot(x, u_ex, "--", label="exact")
plt.title(rf"Numerical vs Exact for eigenvalue #{k}, $h=2\pi/64$")
plt.legend()
plt.grid(True)
plt.show()

```



Question 3(b):

```

[11]: # Setup
N_values = [32, 64, 128, 256, 512, 1024]
h_values = []

```

```

err_2 = []
err_12 = []
err_22 = []

# Results
for kk, N in enumerate(N_values):

    A, h = build_A(N)
    h_values.append(h)
    L = 2.0*np.pi

    vals, vecs = eigs(A, k=24, which="SM")
    idx = np.argsort(np.abs(vals))
    vals = vals[idx]
    vecs = vecs[:, idx]

    # 2nd eigenvalue
    lam_num = np.real(vals[2-1])
    lam_ex = lambda_exact(2)
    err2 = np.abs(abs(lam_num) - abs(lam_ex))
    err_2.append(err2)

    # 12th eigenvalue
    lam_num = np.real(vals[12-1])
    lam_ex = lambda_exact(12)
    err12 = np.abs(abs(lam_num) - abs(lam_ex))
    err_12.append(err12)

    # 22nd eigenvalue
    lam_num = np.real(vals[22-1])
    lam_ex = lambda_exact(22)
    err22 = np.abs(abs(lam_num) - abs(lam_ex))
    err_22.append(err22)

    if kk == 0:
        print(f"n={N}  h={h:.4g}  |err_2|={err2:.6g}  |err_12|={err12:.6g}  |err_22|={err22:.6g}  order=--")
    else:
        rate2 = np.log(err_2[-2]/err_2[-1])/np.log(2.0)
        rate12 = np.log(err_12[-2]/err_12[-1])/np.log(2.0)
        rate22 = np.log(err_22[-2]/err_22[-1])/np.log(2.0)
        print(f"n={N}  h={h:.4g}  |err_2|={err2:.6g}  |err_12|={err12:.6g}  |err_22|={err22:.6g}  order=({rate2:.6f}, {rate12:.6f}, {rate22:.6f})")

# log-log plot
plt.figure()
plt.loglog(h_values, err_2, marker="o", label="2nd eigenvalue")

```



```

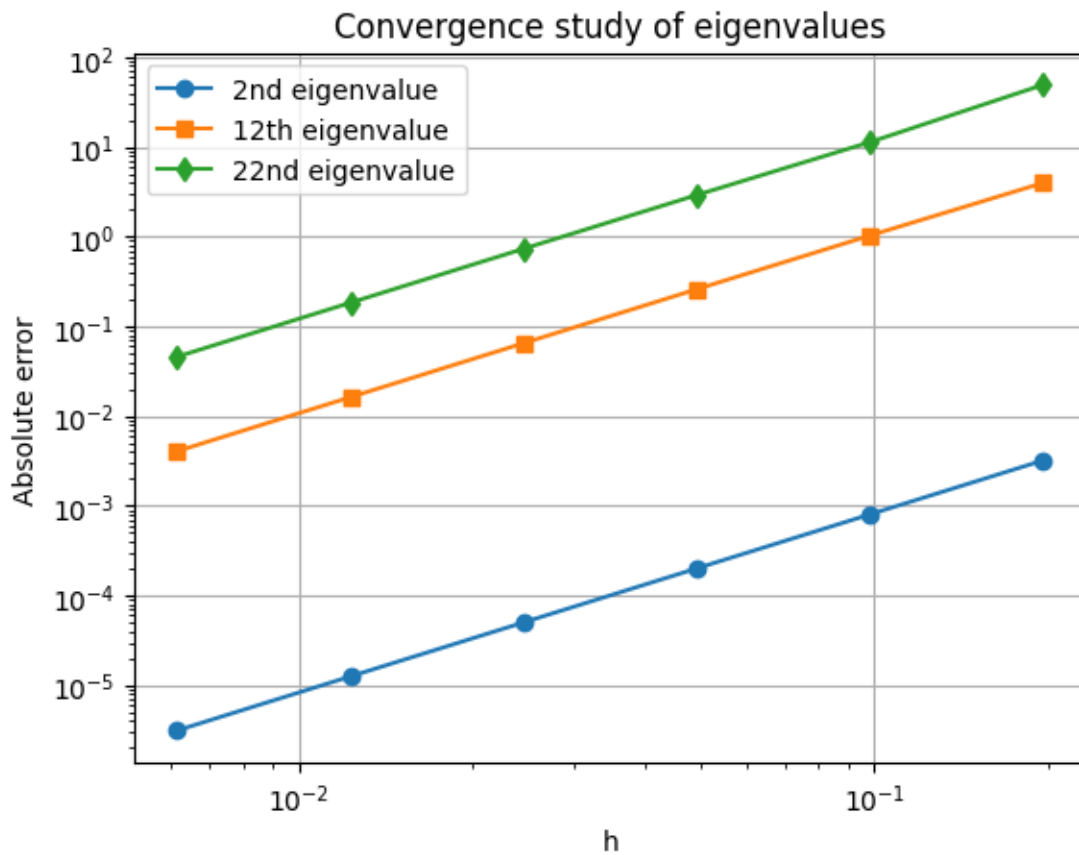
plt.loglog(h_values, err_12, marker="s", label="12th eigenvalue")
plt.loglog(h_values, err_22, marker="d", label="22nd eigenvalue")
plt.xlabel("h")
plt.ylabel("Absolute error")
plt.title("Convergence study of eigenvalues")
plt.grid(True)
plt.legend()
plt.show()

```

```

n=32  h=0.1963  |err_2|=0.00320864  |err_12|=3.97581  |err_22|=49.2713  order=--
n=64  h=0.09817  |err_2|=0.000802932  |err_12|=1.02897  |err_22|=11.3118
order=(1.998610, 1.950048, 2.122923)
n=128  h=0.04909  |err_2|=0.000200781  |err_12|=0.259482  |err_22|=2.91145
order=(1.999652, 1.987492, 1.958012)
n=256  h=0.02454  |err_2|=5.01984e-05  |err_12|=0.0650114  |err_22|=0.733186
order=(1.999913, 1.996872, 1.989489)
n=512  h=0.01227  |err_2|=1.25498e-05  |err_12|=0.0162617  |err_22|=0.183631
order=(1.999978, 1.999218, 1.997371)
n=1024  h=0.006136  |err_2|=3.13746e-06  |err_12|=0.00406597  |err_22|=0.0459286
order=(1.999996, 1.999804, 1.999343)

```



The numerical results show that the errors for the 2nd, 12th, and 22nd eigenvalues all decrease proportionally to h^2 . Hence the method is convergent, and the observed order of accuracy is 2.

Question 3(c):

Correct to 3 significant digits means that the numerical eigenvalue must agree with the true values in the first three digits, which means $|\lambda_{num} - \lambda_{exact}| < 0.5$

```
[15]: N_values = [32, 64, 128, 256, 512, 1024]
print("h                                |error_22|")

for N in N_values:
    A, h = build_A(N)
    L = 2.0*np.pi

    vals, vecs = eigs(A, k=24, which="SM")
    idx = np.argsort(np.abs(vals))
    vals = vals[idx]
    vecs = vecs[:, idx]

    # 22nd eigenvalue
    lam_num = np.real(vals[22-1])
    lam_ex = lambda_exact(22)
    err22 = np.abs(abs(lam_num) - abs(lam_ex))

    print(f"{h:.6e}                {err22:.6f}")
```

h	error_22
1.963495e-01	49.271298
9.817477e-02	11.311765
4.908739e-02	2.911455
2.454369e-02	0.733186
1.227185e-02	0.183631
6.135923e-03	0.045929

Hence, the largest h such that $|\lambda_{num} - \lambda_{exact}| < 0.5$ is between [256, 512]:

```
[31]: N_values = [310, 311]
print("h                                |error_22|")

for N in N_values:
    A, h = build_A(N)
    L = 2.0*np.pi

    vals, vecs = eigs(A, k=24, which="SM")
    idx = np.argsort(np.abs(vals))
    vals = vals[idx]
```

```

vecs = vecs[:, idx]

# 22nd eigenvalue
lam_num = np.real(vals[22-1])
lam_ex = lambda_exact(22)
err22 = np.abs(abs(lam_num) - abs(lam_ex))

print(f"{h}          {err22:.6f}")

```

h	$ \text{error}_{22} $
0.02026833970057931	0.500387
0.02020316819028806	0.497180

Hence, we find when $h \leq 0.02020316819028806$, we have the 22nd eigenvalue correct to three significant digits

MATH405-HW5-Q4

November 26, 2025

Question 4(a):

```
1      format long
2
3      N = 65536;
4      SIN_TABLE = zeros(1, N, "single");
5
6      for i = 1:N
7          angle = (i-1) * (2*pi) / N;
8          SIN_TABLE(i) = single(sin(angle));
9      end
10
11      SIN_TABLE(42)
12
13      |
```

命令行窗口

```
ans =

    single

    0.0039308
```

Question 4(b):

We already use SIN_TABLE, we don't need the COS_TABLE since $\cos(x) = \sin(x + \frac{\pi}{2})$, we only need to do a little shifting.

Question 4(c):

I just run this code and got $s = 0.2419570 \approx \sin(166\text{degrees})$ and $c = -0.9702870 \approx \cos(166\text{degrees})$. Explain:

- $\text{const1} = \text{single}(0.017453292) \approx \frac{\pi}{180}$, so it converts degrees $\rightarrow$ radians
- $\text{const2} = \text{single}(10430.378) \approx \frac{65536}{2\pi}$, which converts radians $\rightarrow$ table index in a table of size 65536.
- Hence, we say $s = \sin(\text{yaw})$ and $c = \cos(\text{yaw})$, this code converts the yaw angle from degrees into the index and returns single-precision approximations of $\sin(\text{yaw})$ and $\cos(\text{yaw})$ using that sine table.

Question 5(d):

```

13 yaw = single(166.02812)
14
15 const1 = single(0.017453292)
16 const2 = single(10430.378)
17 value = yaw * const1
18 % note: rem is remainder, int64 converts to integer
19 % the exact placement of brackets here matters
20 sindex = rem(int64(value * const2), N) + 1
21 cindex = rem(int64(value * const2 + 16384), N) + 1
22
23 s = SIN_TABLE(sindex)
24 c = SIN_TABLE(cindex)
25
26 result = c^2 + s^2 - 1
27

```

命令行窗口

result =

single

0

Question 4(e):

```

14 yaw = single(5850000);
15
16 best_val = single(0);
17 best_yaw = yaw;
18
19 steps = 1000000;
20
21 for k= 1:steps
22
23     value = yaw * const1;
24     sindex = rem(int64(value * const2), N) + 1;
25     cindex = rem(int64(value * const2 + 16384), N) + 1;
26
27     s = SIN_TABLE(sindex);
28     c = SIN_TABLE(cindex);
29
30     val = c^2 + s^2
31
32     if val > best_val
33         best_val = val;
34         best_yaw = yaw;
35     end

```

```

36
37     yaw = yaw + eps(yaw); % next single-precision float
38 end
39
40 speed_factor = sqrt(double(best_val));
41 percent = (speed_factor - 1.0) * 100;
42
43 best_yaw
44 best_val
45 speed_factor
46 percent

```



```
best_yaw =  
    single  
    5898195  
  
best_val =  
    single  
    1.0061358  
  
speed_factor =  
    1.003063219015865  
  
percent =  
    0.306321901586459
```

Question 4(f):

For example, “cindex = rem(int64(value * (const2 + 16384)), N) + 1” → “cindex = rem(int64(value * const2 + 16384), N) + 1”. This breaks the identity $\cos(x) = \sin(x + \frac{\pi}{2})$, so the cindex no longer points to a value exactly 16384 entries after sindex.

MATH405-HW5-Q5

November 26, 2025

Question 5(a):

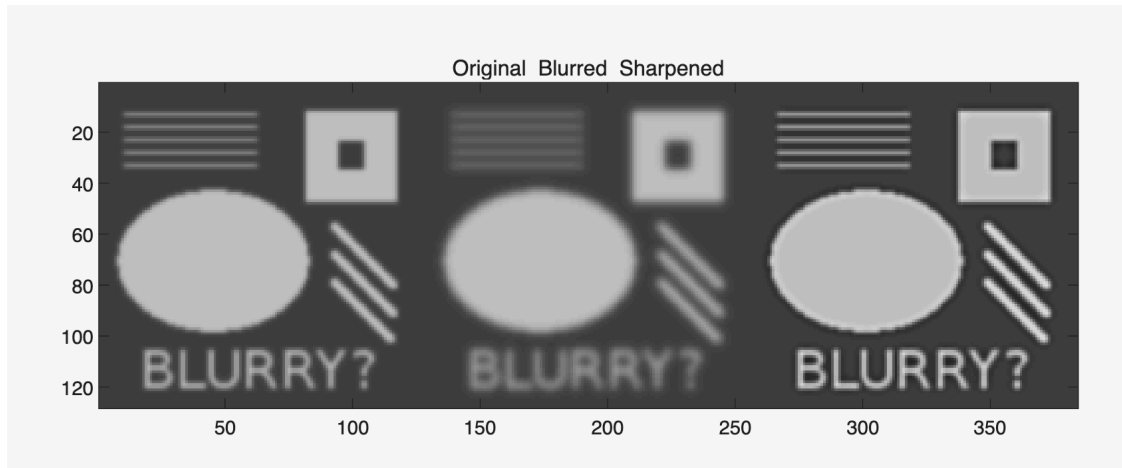
```
1  %% Read an image from a file
2  % Notice I write the code below according to the code provided by last
3  % homework sol.
4  u = imread('testpat_blur2.png');
5  u = double(u)/255;
6
7  [n,n2] = size(u);
8  if (n ~= n2)
9      error('by defalut, this only supports square images');
10 end
11
12 figure(1); clf;
13 imagesc(u);
14 caxis([0 1]);
15 colormap(gray);
16 axis image
17 title('Original Image')
18
19 %% Build discrete Laplacian matrix (same as previous homework)
20 e = ones(n,1);
21 L1 = spdiags([e -2*e e], [-1 0 1], n, n);
22 % Periodic BCs
23 L1(1,1) = -1;
24 L1(end,end) = -1;
25 I = speye(n,n);
26 L = kron(L1,I) + kron(I,L1);
27
28 %% "stretch" the matrix
29 v = reshape(u, n*n, 1);
30
31 k = 0.1; % time-step
32 steps = 10; % number of blur steps
```

```

34     for i = 1:steps
35         v=v + k * (L*v);
36     end
37
38     u_blur = reshape(v, n, n);
39
40     figure(2); clf;
41     imagesc(u_blur);
42     caxis([0 1]);
43     colormap(gray);
44     axis image
45     title('Blurred Image')
46
47     %% Unsharp Mask
48     edge_map = u - u_blur;
49     alpha = 1.0; % strength of sharpening
50
51     u_sharp = u + alpha * edge_map;
52
53     figure(3); clf;
54     imagesc(u_sharp);
55     caxis([0 1]);
56     colormap(gray);
57     axis image
58     title('Unsharp Mask Image')
59
60
61     %% Show all side-by-side
62     result = [u u_blur u_sharp];
63     figure(4);
64     clf;
65     imagesc(result);
66
67     %% Show all side-by-side
68     result = [u u_blur u_sharp];
69     figure(4);
70     clf;
71     imagesc(result);
72     caxis([0 1]);
73     colormap(gray);
74     axis image
75     title('Original Blurred Sharpened')

```

For example, if we input testpat_blur2.png:



Question 5(b):

I have already provided the results applied to testpat_blur2.png in part (a). Next, I will only show the modified code and results for the zoom-in section.

```

47  %% Unsharp Mask
48  edge_map = u - u_blur;
49  alpha = 1.0; % strength of sharpening
50
51  u_sharp = u + alpha * edge_map;
52
53  figure(3); clf;
54  imagesc(u_sharp(70:end, 50:end));
55  caxis([0 1]);
56  colormap(gray);
57  axis image
58  title('Zoomed-in unsharp')
59
60  %% Find the lightest and darkest pixel values
61  umin = min(u_sharp(:));
62  umax = max(u_sharp(:));
63
64  fprintf('min pixel = %.4f, max pixel = %.4f\n', umin, umax);
65

```

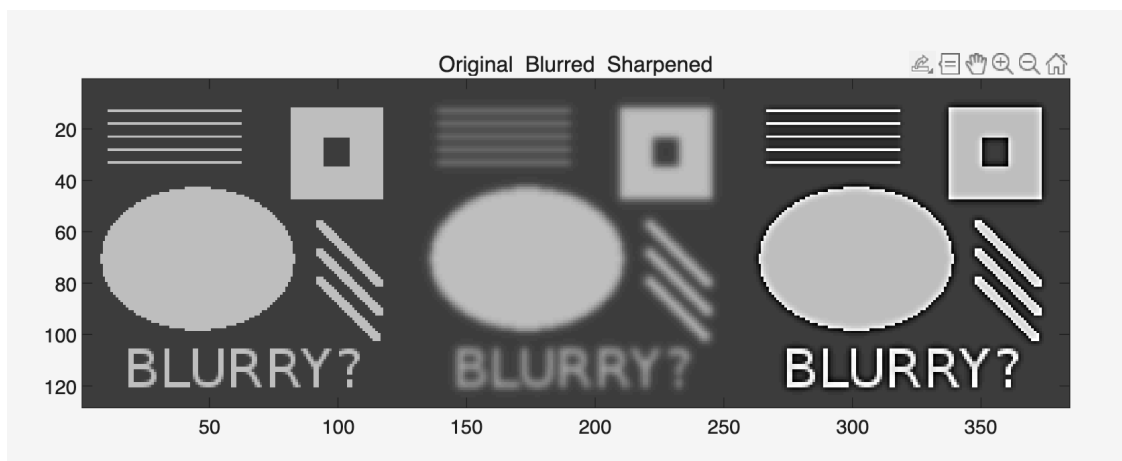
Results:



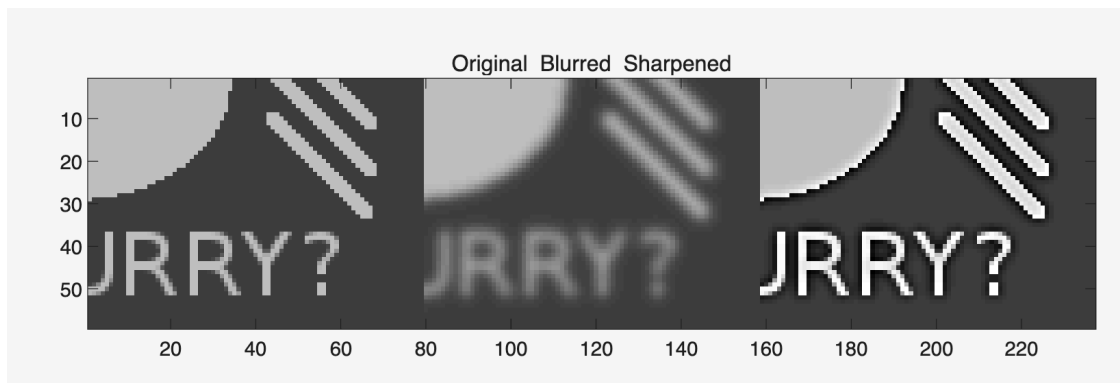
and min pixel = 0.1470, max pixel = 0.8737.

Question 5(c):

For testpat_noblur.png:

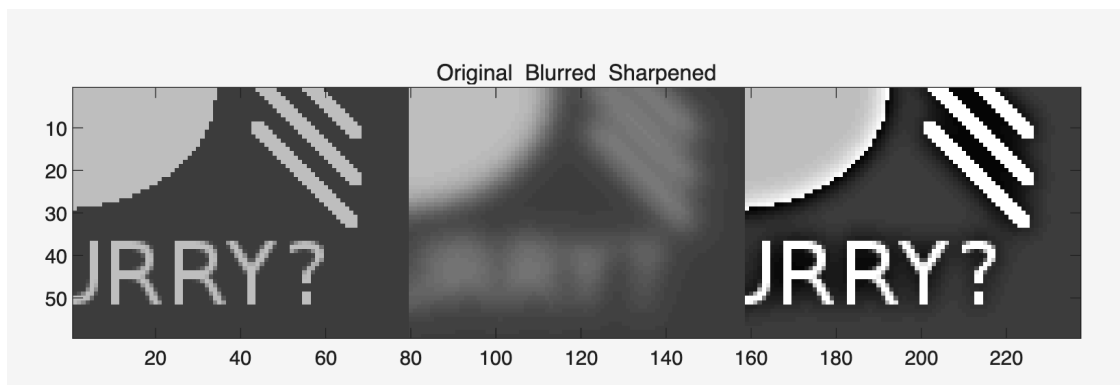


After zoom in:



Comment: so if the image is already sharp, the unsharp mask locally boosts contrast at edges, making bright edge pixels brighter and dark edge pixels darker, while leaving flat region unchanged.

“The number of blurring steps” means “steps” in my code, I initially set steps=10, let us set it as 50 to see what gonna happened (for testpat_noblur.png and compare with the zoo-in plot above):



How about decrease? For example, steps=2:

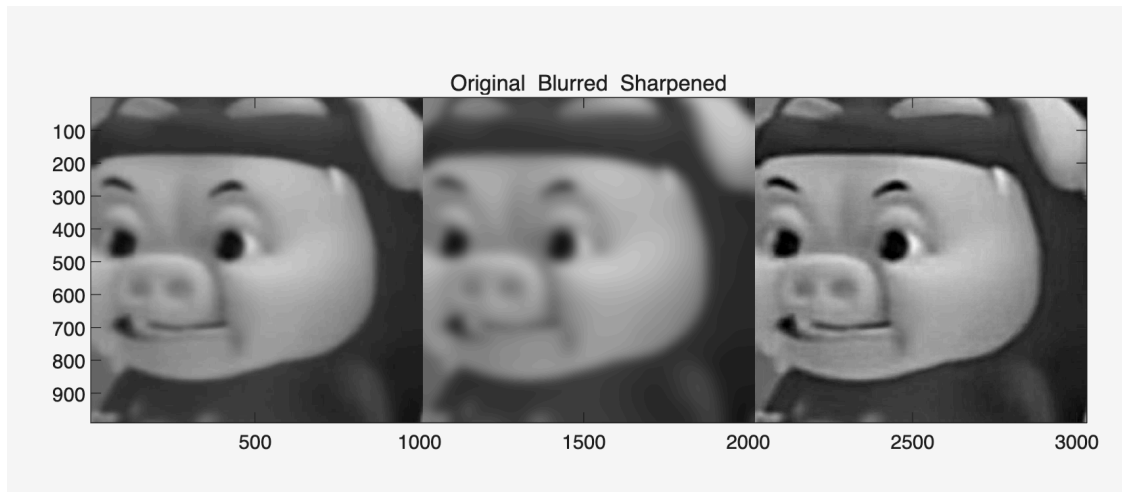


Hence, we found: Increasing the number of blurring steps makes the blurred image more blurred, which produces a larger edge map and therefore a stronger sharpening effect; fewer steps give weaker sharpening.

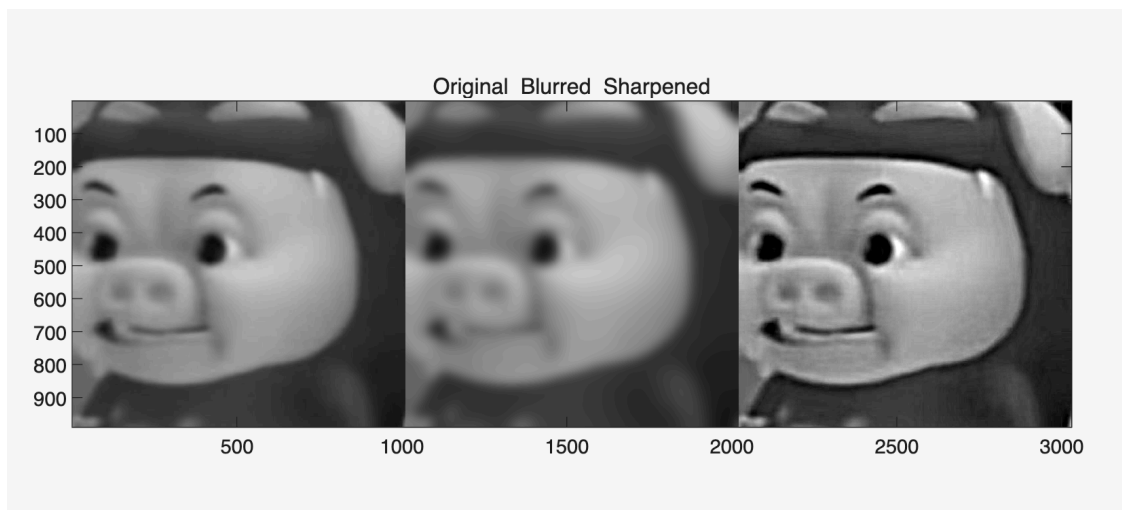
Question 5(d):

I used an anime pig image to show how changing the alpha affects sharpening.

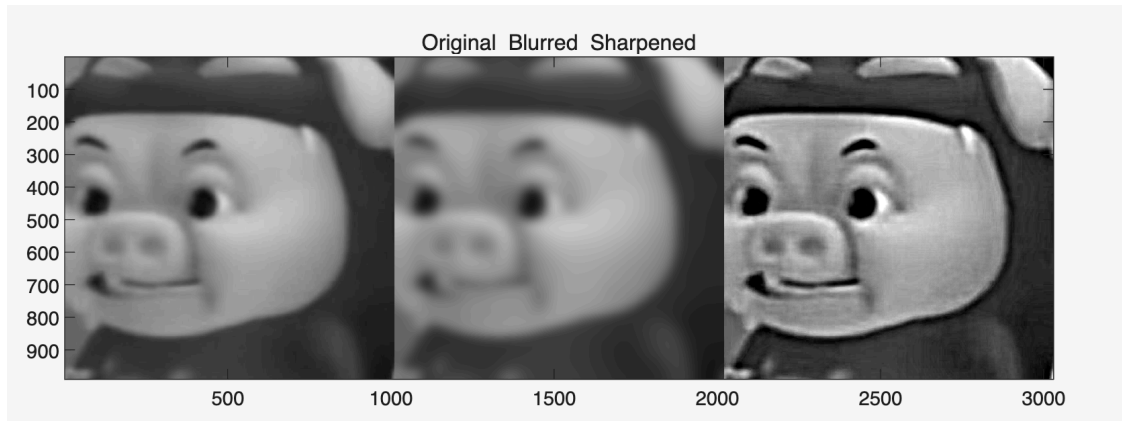
If $\alpha = 1$ and steps=1000:



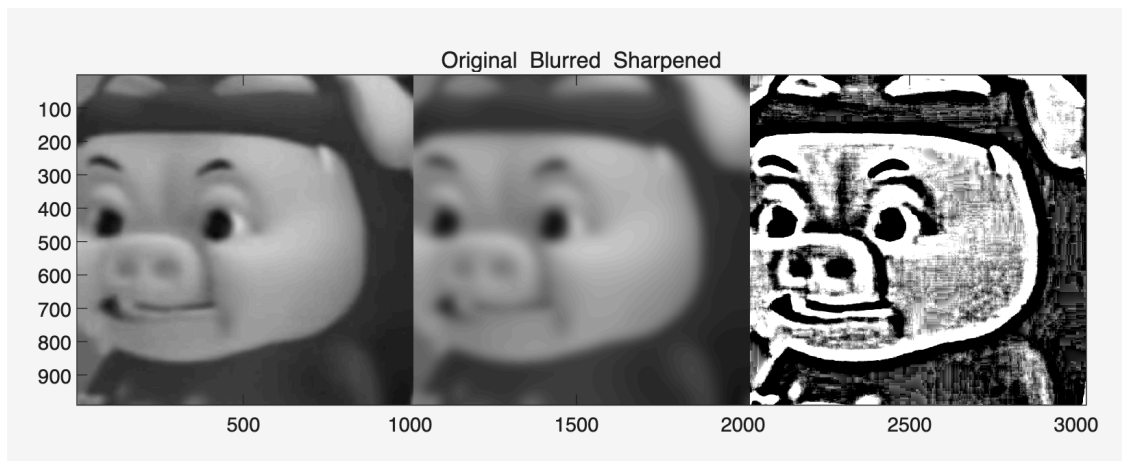
If $\alpha = 2$ and steps=1000:



If $\alpha = 3$ and steps=1000:



If $\alpha = 50$ and steps=1000:



Comment: For the unsharp mask

$$u_{sharp} = u + \alpha(u - u_{blur})$$

, the parameter α directly controls how strongly the edge map. Under normal situation, increasing α will strengthen the sharpening. However, once alpha is large beyond a certain bound, it produces counterproductive results. This causes the images at edges—dark areas become overdark while light areas become overbright, resulting in distorted (and even scary) images.